

Progress Report

Day 10 — MySQL Audit Logging Integration

Task Management MERN Stack Platform

Shafin Nigamana | 28 May 2026

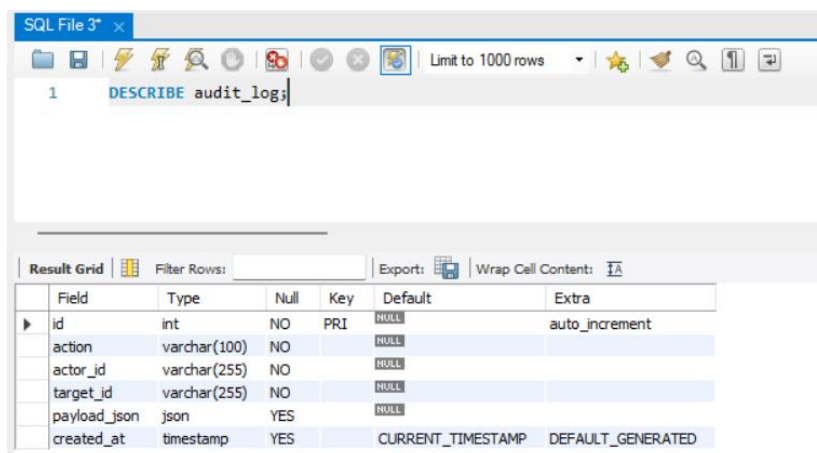
1. Day-10 Objective

The goal for Day 10 was to implement MySQL-based audit logging for Task operations and integrate automatic activity tracking into the backend Task management workflow.

2. Tasks Completed

MySQL Audit Table Design

- Created audit_log table inside MySQL database
- Configured columns:
 - id
 - action
 - actor_id
 - target_id
 - payload_json
 - created_at
- Verified table structure using MySQL queries



The screenshot shows a MySQL IDE window titled "SQL File 3". The query editor contains the command `DESCRIBE audit_log;`. Below the editor, the "Result Grid" displays the table structure. The table has six columns: id, action, actor_id, target_id, payload_json, and created_at. The 'id' column is an integer, primary key, and auto-increment. The 'action', 'actor_id', and 'target_id' columns are VARCHAR(100), VARCHAR(255), and VARCHAR(255) respectively. The 'payload_json' column is JSON. The 'created_at' column is a timestamp with a default value of CURRENT_TIMESTAMP and is generated by default.

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
action	varchar(100)	NO		NULL	
actor_id	varchar(255)	NO		NULL	
target_id	varchar(255)	NO		NULL	
payload_json	json	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Figure 2.1 — MySQL audit_log table structure verified successfully.

Audit Logger Utility

- Created reusable auditLogger.js utility inside server/src/utlis
- Implemented centralized logAudit() helper for MySQL audit inserts
- Configured JSON payload serialization using JSON.stringify()
- Added safe error handling to prevent backend crashes during audit failures

```

server > src > utlis > JS auditLogger.js > ...
1  import { createMySQLPool } from '../config/mysql.js';
2
3  let isTableInitialized = false;
4
5  const initAuditTable = async (pool) => {
6    if (isTableInitialized) return;
7
8    try {
9      const query = `
10       CREATE TABLE IF NOT EXISTS audit_log (
11         id INT AUTO INCREMENT PRIMARY KEY,
12         action VARCHAR(100) NOT NULL,
13         actor_id VARCHAR(255) NOT NULL,
14         target_id VARCHAR(255) NOT NULL,
15         payload_json JSON,
16         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
17       );
18     `;
19     await pool.query(query);
20
21     isTableInitialized = true;
22   } catch (error) {
23     console.error('Failed to initialize audit_log table:', error.message);
24   }
25 };
26
27 /**
28  * Logs an audit event to the MySQL database.
29  * Fails safely without throwing errors.
30  *
31  * @param {string} action - The action performed (e.g. 'CREATE TASK')
32  * @param {string} actorId - ID of the user performing the action
33  * @param {string} taskId - ID of the task affected
34  * @param {object} payload - Relevant payload data snapshot
35  */
36 export const logAudit = async (action, actorId, taskId, payload) => {
37   try {
5  const initAuditTable = async (pool) => {
6    } catch (error) {
7      console.error('Failed to initialize audit_log table:', error.message);
8    }
9  };
10
11  /**
12   * Logs an audit event to the MySQL database.
13   * Fails safely without throwing errors.
14   *
15   * @param {string} action - The action performed (e.g. 'CREATE TASK')
16   * @param {string} actorId - ID of the user performing the action
17   * @param {string} taskId - ID of the task affected
18   * @param {object} payload - Relevant payload data snapshot
19   */
20  export const logAudit = async (action, actorId, taskId, payload) => {
21    try {
22      const pool = createMySQLPool();
23      await initAuditTable(pool);
24
25      const query = `
26        INSERT INTO audit_log (action, actor_id, target_id, payload_json)
27        VALUES (?, ?, ?, ?)
28      `;
29
30      const safePayload = payload ? JSON.stringify(payload) : null;
31      const safeActorId = actorId || 'SYSTEM';
32      const safeTaskId = taskId || 'UNKNOWN';
33
34      await pool.execute(query, [action, safeActorId, safeTaskId, safePayload]);
35    } catch (error) {
36      console.error('Audit Logging Failed [${action}]:', error.message);
37    }
38  };

```

Figure 2.2 — Reusable audit logging utility implementation.

Task Controller Audit Integration

- Integrated audit logging into:
 - createTask
 - updateTask
 - deleteTask
- Configured audit entries to log:
 - action type
 - authenticated user ID

- related task ID
- payload snapshot

```

1  import Task from '../models/Task.js';
2  import { logAudit } from '../utils/auditLogger.js';
3
4  export const createTask = async (req, res) => {
5    try {
6      const { title, description, assigneeId, teamId, status, priority, dueDate } = req.body || {};
7
8      if (!title) {
9        return res.status(400).json({ message: 'Task title is required' });
10     }
11
12     const task = await Task.create({
13       title,
14       description,
15       assigneeId,
16       teamId,
17       status,
18       priority,
19       dueDate,
20     });
21
22     // Log audit event asynchronously
23     logAudit('CREATE_TASK', req.user?.id, task._id.toString(), {
24       title: task.title,
25       status: task.status
26     });
27
28     return res.status(201).json(task);
29   } catch (error) {
30     if (error.name === 'ValidationError') {
31       return res.status(400).json({ message: 'Invalid task data', error: error.message });
32     }
33     return res.status(500).json({ message: 'Error creating task', error: error.message });
34   }
35 };
36
37 export const getTasks = async (req, res) => {
38   try {
39     const { status, priority, assigneeId, teamId } = req.query;
40     const filter = {};
41
42     // Basic dynamic filtering based on query params
43     if (status) filter.status = status;
44     if (priority) filter.priority = priority;
45     if (assigneeId) filter.assigneeId = assigneeId;
46     if (teamId) filter.teamId = teamId;
47
48     const tasks = await Task.find(filter);
49     return res.status(200).json(tasks);
50   } catch (error) {
51     return res.status(500).json({ message: 'Error fetching tasks', error: error.message });
52   }
53 };
54
55 export const getTaskById = async (req, res) => {
56   try {
57     const { id } = req.params;
58     const task = await Task.findById(id);
59
60     if (!task) {
61       return res.status(404).json({ message: 'Task not found' });
62     }
63
64     return res.status(200).json(task);
65   } catch (error) {
66     return res.status(500).json({ message: 'Error fetching task', error: error.message });
67   }
68 };
69
70 export const updateTask = async (req, res) => {
71   try {
72     const { id } = req.params;
73     const updates = req.body || {};
74
75     const task = await Task.findByIdAndUpdate(
76       id,
77       updates,
78       { new: true, runValidators: true }
79     );
80
81     if (!task) {
82       return res.status(404).json({ message: 'Task not found' });
83     }
84
85     // Log audit event asynchronously
86     logAudit('UPDATE_TASK', req.user?.id, task._id.toString(), {
87       updatedFields: updates
88     });
89
90     return res.status(200).json(task);
91   } catch (error) {
92     if (error.name === 'ValidationError') {
93       return res.status(400).json({ message: 'Invalid task data', error: error.message });
94     }
95     return res.status(500).json({ message: 'Error updating task', error: error.message });
96   }
97 };
98
99 export const deleteTask = async (req, res) => {
100  try {
101    const { id } = req.params;
102    const task = await Task.findByIdAndDelete(id);
103
104    if (!task) {
105      return res.status(404).json({ message: 'Task not found' });
106    }
107
108    // Log audit event asynchronously
109    logAudit('DELETE_TASK', req.user?.id, id, {
110      deletedFields: task
111    });
112  }

```

Figure 2.3 — Task controller integrated with MySQL audit logging flow.

CREATE_TASK Audit Verification

- Tested task creation endpoint using Postman
- Verified backend automatically inserts CREATE_TASK entry into MySQL audit table after successful MongoDB operation

```

1  SELECT * FROM audit_log
2  WHERE action = 'CREATE_TASK'
3  ORDER BY created_at DESC;

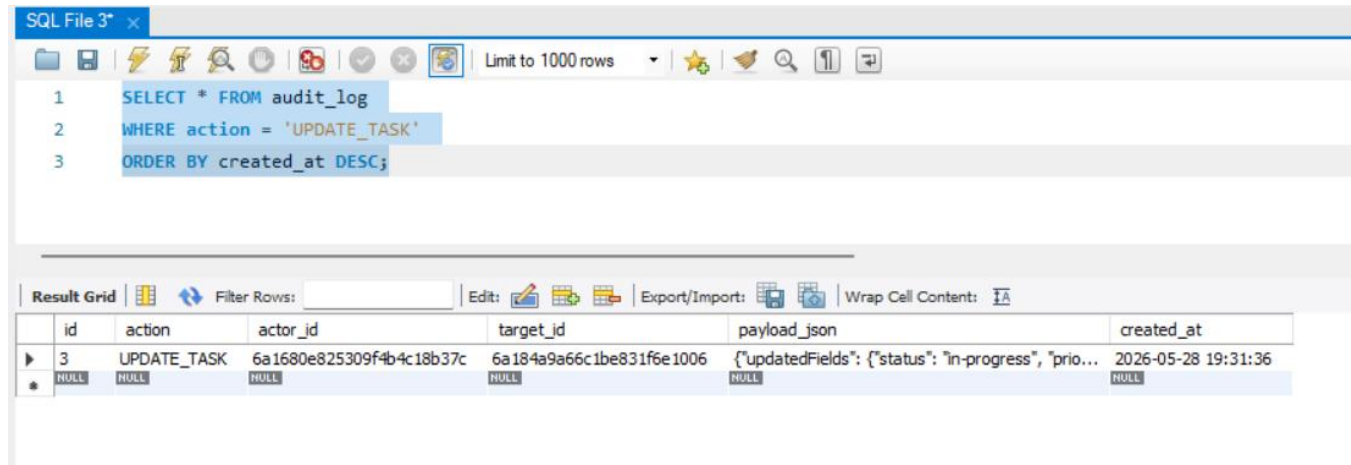
```

	id	action	actor_id	target_id	payload_json	created_at
▶	1	CREATE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"title": "Design API", "status": "todo"}	2026-05-28 19:30:58
	2	CREATE_TASK	6a1680e825309f4b4c18b37c	6a1849c266c1be831f6e1005	{"title": "Build Audit System", "status": "todo"}	2026-05-28 19:27:22
*						

Figure 2.4 — CREATE_TASK audit entry stored successfully in MySQL.

UPDATE_TASK Audit Verification

- Tested task update endpoint using Postman
- Verified backend automatically inserts UPDATE_TASK audit entry after successful task modification



The screenshot shows a SQL client interface with a query window and a result grid. The query is:

```
1 SELECT * FROM audit_log
2 WHERE action = 'UPDATE_TASK'
3 ORDER BY created_at DESC;
```

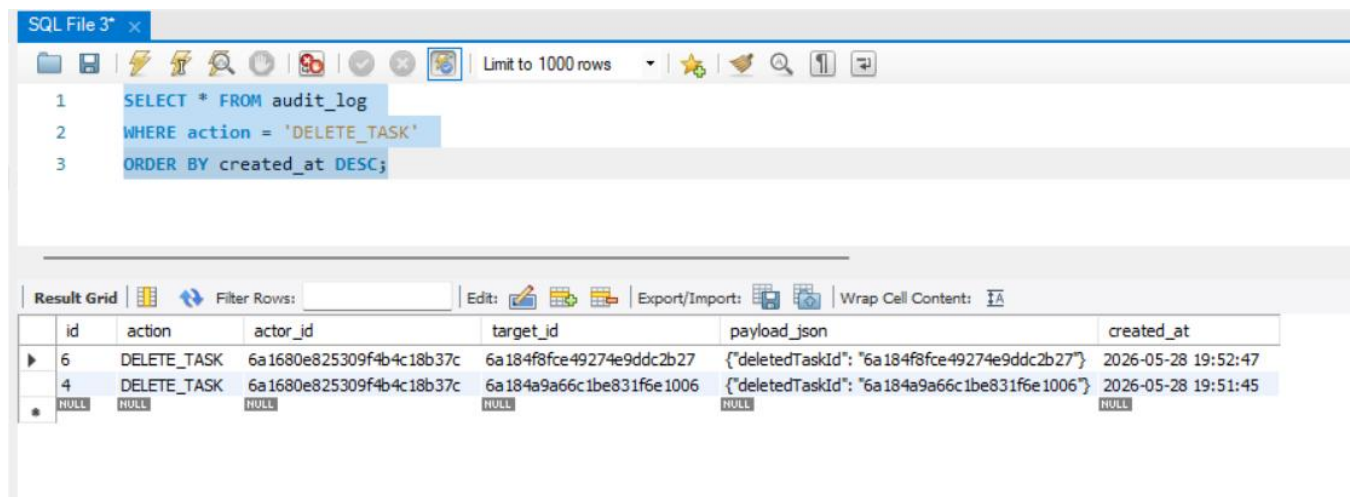
The result grid displays one row of data:

	id	action	actor_id	target_id	payload_json	created_at
▶	3	UPDATE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"updatedFields": {"status": "in-progress", "prio...	2026-05-28 19:31:36

Figure 2.5 — UPDATE_TASK audit entry verified successfully inside MySQL.

DELETE_TASK Audit Verification

- Tested task delete endpoint using Postman
- Verified backend automatically inserts DELETE_TASK audit entry after task removal



The screenshot shows a SQL client interface with a query window and a result grid. The query is:

```
1 SELECT * FROM audit_log
2 WHERE action = 'DELETE_TASK'
3 ORDER BY created_at DESC;
```

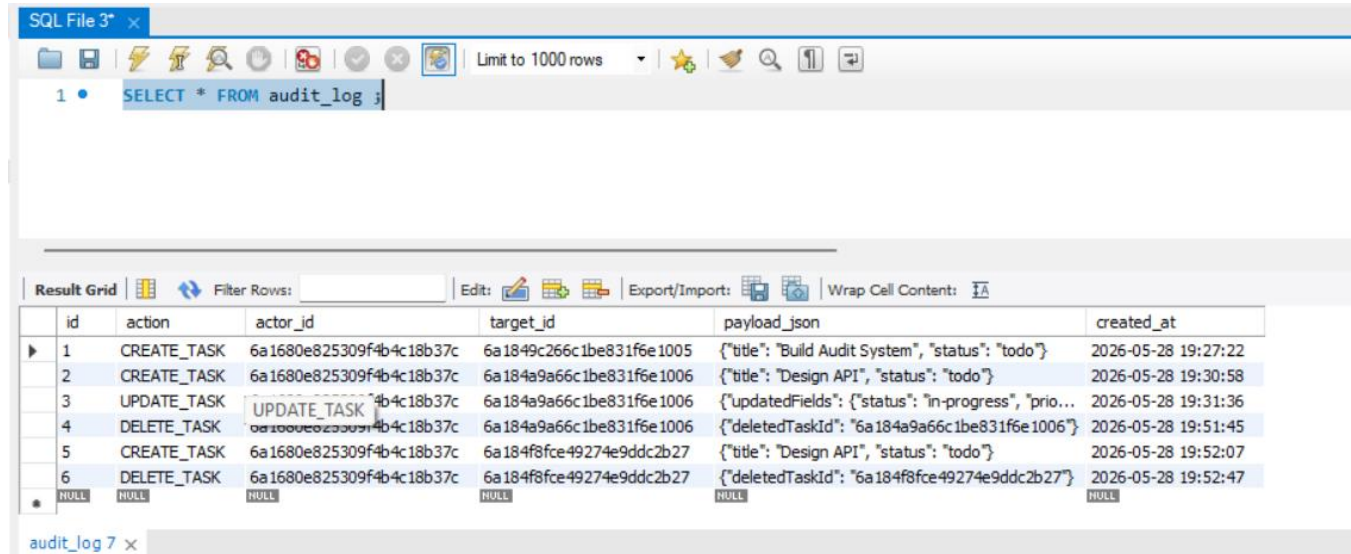
The result grid displays two rows of data:

	id	action	actor_id	target_id	payload_json	created_at
▶	6	DELETE_TASK	6a1680e825309f4b4c18b37c	6a184f8fce49274e9ddc2b27	{"deletedTaskId": "6a184f8fce49274e9ddc2b27"}	2026-05-28 19:52:47
	4	DELETE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"deletedTaskId": "6a184a9a66c1be831f6e1006"}	2026-05-28 19:51:45

Figure 2.6 — DELETE_TASK audit entry written successfully into MySQL.

Complete Audit History Verification

- Verified multiple audit entries are stored correctly inside MySQL audit_log table
- Confirmed payload_json stores serialized task activity data successfully



The screenshot shows a SQL client interface with a query window and a result grid. The query window contains the SQL statement: `SELECT * FROM audit_log ;`. The result grid displays the following data:

	id	action	actor_id	target_id	payload_json	created_at
▶	1	CREATE_TASK	6a1680e825309f4b4c18b37c	6a1849c266c1be831f6e1005	{"title": "Build Audit System", "status": "todo"}	2026-05-28 19:27:22
	2	CREATE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"title": "Design API", "status": "todo"}	2026-05-28 19:30:58
	3	UPDATE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"updatedFields": {"status": "in-progress", "prio...	2026-05-28 19:31:36
	4	DELETE_TASK	6a1680e825309f4b4c18b37c	6a184a9a66c1be831f6e1006	{"deletedTaskId": "6a184a9a66c1be831f6e1006"}	2026-05-28 19:51:45
	5	CREATE_TASK	6a1680e825309f4b4c18b37c	6a184f8fce49274e9ddc2b27	{"title": "Design API", "status": "todo"}	2026-05-28 19:52:07
	6	DELETE_TASK	6a1680e825309f4b4c18b37c	6a184f8fce49274e9ddc2b27	{"deletedTaskId": "6a184f8fce49274e9ddc2b27"}	2026-05-28 19:52:47
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 2.7 — Complete audit history verified successfully using MySQL audit_log table.

Repository Update

- Committed audit logging implementation changes using Git workflow
- Pushed latest backend audit integration updates to GitHub repository

```

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git status
On branch feature/task-module
Your branch is up to date with 'origin/feature/task-module'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   server/src/config/mysql.js
    modified:   server/src/controllers/taskController.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    server/src/utils/

no changes added to commit (use "git add" and/or "git commit -a")

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git add .

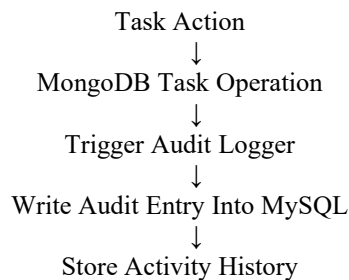
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git commit -m "feat: implement MySQL audit logging for task actions"
[feature/task-module 2ccc681] feat: implement MySQL audit logging for task actions
3 files changed, 72 insertions(+), 1 deletion(-)
create mode 100644 server/src/utils/auditLogger.js

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/task-module)
$ git push
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 1.74 KiB | 222.00 KiB/s, done.
Total 10 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To github.com:ShafinNigamana/task-management-mern.git
   0f9ec92..2ccc681  feature/task-module -> feature/task-module

```

Figure 2.8 — Day 10 audit logging implementation pushed to GitHub.

3. Current Audit Flow



4. Next Steps (Day 11)

- Implement `/api/audit` endpoint
- Add paginated audit history retrieval

- Verify audit log API responses using Postman
- Document backend API endpoints in README

5. Conclusion

Day 10 focused on implementing MySQL audit logging and integrating backend activity tracking into the Task management workflow. Audit entries for task creation, updates, and deletion were verified successfully using MySQL queries and Postman testing.

The backend now supports dual-database architecture with MongoDB handling operational data and MySQL handling audit history tracking.